



Local Life Project 환경 설정 및 실행 방법

환경 설정

1) 필수 환경 변수

환경 변수는 `application.properties` 또는 OS 환경 변수로 설정합니다.

다음은 이름만 예시이며, 실제 값은 레포에 커밋하지 않고 별도 관리해야 합니다.

- DB
 - `DB_URL`
 - `DB_USER`
 - `DB_PASSWORD`
 - `DDL_AUTO`
- JWT
 - `JWT_SECRET`
- Redis
 - `SPRING_DATA_REDIS_HOST`
 - `SPRING_DATA_REDIS_PORT`
 - `SPRING_DATA_REDIS_PASSWORD`
- AWS S3
 - `AWS_ACCESS_KEY`
 - `AWS_SECRET_KEY`
 - `AWS_REGION`
 - `AWS_S3_BUCKET`

- Mail
 - MAIL_USERNAME
 - MAIL_PASSWORD
- OpenAI
 - OPENAI_API_KEY
- PortOne (결제)
 - IAMPORT_API_KEY
 - IAMPORT_API_SECRET_KEY

실제 키 값은 .env 환경 변수로만 관리해야 합니다.

2) application.properties 예시

```
spring.application.name=local-life-backend
spring.config.import=optional:file:.env[.properties]

# DB
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}

# JPA & Hibernate
spring.jpa.database=mysql
spring.jpa.hibernate.ddl-auto=${DDL_AUTO}
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Redis
spring.data.redis.host=${SPRING_DATA_REDIS_HOST}
spring.data.redis.port=${SPRING_DATA_REDIS_PORT}
spring.data.redis.password=${SPRING_DATA_REDIS_PASSWORD}

# JWT
jwt.secret=${JWT_SECRET}
```

```
jwt.access-exp=7200000    # 2시간 (ms)
jwt.refresh-exp=259200000  # 3일 (ms)

# AWS S3
cloud.aws.credentials.access-key=${AWS_ACCESS_KEY}
cloud.aws.credentials.secret-key=${AWS_SECRET_KEY}
cloud.aws.region.static=${AWS_REGION}
cloud.aws.s3.bucket=${AWS_S3_BUCKET}
cloud.aws.stack.auto=false

# Profile
spring.profiles.active=default

# Flyway (검색 인덱스용)
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.validate-on-migrate=true
spring.flyway.baseline-on-migrate=true
```

JPA & Flyway 사용 방식 안내

본 프로젝트는 **JPA(Hibernate)**를 중심으로 스키마를 관리합니다.

Flyway는 프로그램 검색 기능을 위한 **FULLTEXT INDEX** 및 **Ngram** 설정을 적용하기 위해 보조적으로 사용됩니다.

- 초기 테이블 생성 및 기본 스키마 변경
→ **JPA (ddl-auto)**가 담당
- 검색 성능 개선을 위한 인덱스 설정
→ **Flyway 마이그레이션으로 적용**

일반적인 실행 방식

- 기존 DB를 유지한 상태에서 실행하는 경우
 - `spring.flyway.enabled=true`
 - 애플리케이션 실행 시 Flyway가 검색 관련 인덱스를 자동 적용합니다.

DB를 전체 삭제 후 새로 생성하는 경우

DB를 완전히 삭제(drop)한 상태에서는 Flyway가 JPA보다 먼저 실행되며, 인덱스를 적용할 테이블이 없어 오류가 발생할 수 있습니다.

이 경우 아래 순서로 실행합니다.

1. 최초 1회 실행

- `spring.flyway.enabled=false`
- JPA가 Entity 기반으로 테이블 생성

2. 테이블 생성 확인 후 재실행

- `spring.flyway.enabled=true`
- FULLTEXT INDEX 및 Ngram 설정 적용

본 프로젝트는 제한된 기간 내 기능 구현을 우선하기 위해 JPA 기반 스키마 관리 + Flyway 부분 적용 방식을 선택했습니다.

향후 Flyway로 스키마 전체를 관리하는 구조로 전환하는 경우에는 `spring.jpa.hibernate.ddl-auto=validate` 설정을 통해 JPA가 스키마를 변경하지 않도록 구성하는 것이 적절합니다.

실행 방법

1) Repository Clone

```
git clone <https://github.com/salayo-team/local-life-backend.git>
cd local-life-backend
```

2) 환경 변수 / 설정 파일 준비

```
cp src/main/resources/application-example.properties \
src/main/resources/application.properties
```

- 이후 `application.properties` 에 실제 환경 변수 참조 또는 로컬 개발용 값을 입력합니다.

3) 빌드 & 실행

```
./gradlew clean build
./gradlew bootRun
```

```
# 또는  
java -jar build/libs/local-life-backend-*.jar
```